

Modern Storages and Data Warehousing Week 2 - HA и MPP

Попов Илья, i.popov@hse.ru

1 - Отказоустойчивость СУБД

Репликация

- › Мы хотим разгрузить нашу базу
 - Оставить write нагрузку на мастере
 - Увести аналитиков с read нагрузкой (и блокировками на чтение) куда-нибудь подальше от прода
- › Хочется, чтобы при смерти основного instance БД (master) сервис не умирал, а переключался на резервный instance БД (hot standby replica)

Репликация

- › Мы хотим разгрузить нашу базу
 - Оставить write нагрузку на мастере
 - Увести аналитиков с read нагрузкой (и блокировками на чтение) куда-нибудь подальше от прода
- › Хочется, чтобы при смерти основного instance БД (master) сервис не умирал, а переключался на резервный instance БД (hot standby replica)

Для этого у нас существует такой концепт как **репликация**.

Как работает репликация

- › СУБД пишет WAL (Write Ahead Log) - последовательный лог изменений, происходящих с БД
- › В контексте PostgreSQL мы будем говорить про replication_slot - offset (LSN) в журнале, до какого момента его вычитал потребитель
- › replication_slot следит за последним прочитанным LSN и гарантирует, что мастер не удалит WAL, пока реплика его не забрала

Как работает репликация

- › СУБД пишет WAL (Write Ahead Log) - последовательный лог изменений, происходящих с БД
- › В контексте PostgreSQL мы будем говорить про replication_slot - offset (LSN) в журнале, до какого момента его вычитал потребитель
- › replication_slot следит за последним прочитанным LSN и гарантирует, что мастер не удалит WAL, пока реплика его не забрала
- › Репликация бывает **логическая** и **физическая**
- › Репликация бывает **синхронная** и **асинхронная**

Виды репликации

	Физическая	Логическая
Что передаём?	Байты WAL	Логические команды (insert, delete, update)
Гранулярность	Вся БД (или кластер) целиком	Отдельные таблицы
Реплика доступна	Только на чтение	Полноценная база на запись
DDL	Переносится	Не переносится
Последовательности	Переносятся	Не переносятся
Накладные расходы	Ниже	Выше
Сценарии использования	HA, hot standby	Миграции, интеграции, CDC

Виды репликации

async: COMMIT —► клиенту "ок" —► ... —► реплика
быстро. потеряем последние транзакции, если мастер умрёт



sync: COMMIT —► реплика подтвердила —► клиенту "ок"
не потеряем ничего. платим сетевым round-trip за каждую запись



Выбираем, что для нашего сервиса дороже — потерянная транзакция или +2 мс на каждую запись?

Демо 0-2

Чего нам не хватает до полноценной отказоустойчивости?

- › Хотим, чтобы базу нельзя было задудосить кучей открытых подключений
- › Хотим, чтобы мастер сам переключался при смерти одного хоста
- › Хотим, чтобы наши запросы всегда шли в живой хост

Чего нам не хватает до полноценной отказоустойчивости?

- › Хотим, чтобы базу нельзя было задудосить кучей открытых подключений
- › Хотим, чтобы мастер сам переключался при смерти одного хоста
- › Хотим, чтобы наши запросы всегда шли в живой хост

- › Для всего этого есть инструменты!

Нельзя задудосить кучей открытых подключений

- › Q: Почему ``max_connections`` не решает проблему?
- › A: Каждое подключение в PostgreSQL - отдельный процесс ОС со своим ``work_mem``. 1000 подключений == 1000 процессов.

Нельзя задудосить кучей открытых подключений

- › Q: Почему ``max_connections`` не решает проблему?
- › A: Каждое подключение в PostgreSQL - отдельный процесс ОС со своим ``work_mem``. 1000 подключений == 1000 процессов.
- › Здесь мы садим **pgbouncer**
- › Он работает как прокси - открывает указанное количество подключений к базе и передает входящие в него запросы в одно из этих подключений

Нельзя задудосить кучей открытых подключений

- › Q: Почему ``max_connections`` не решает проблему?
- › A: Каждое подключение в PostgreSQL - отдельный процесс ОС со своим ``work_mem``. 1000 подключений == 1000 процессов.
- › Здесь мы садим **pgbouncer**
- › Он работает как прокси - открывает указанное количество подключений к базе и передает входящие в него запросы в одно из этих подключений
- › Q: что тут может разломаться?

Нельзя задудосить кучей открытых подключений

- › Q: Почему ``max_connections`` не решает проблему?
- › A: Каждое подключение в PostgreSQL - отдельный процесс ОС со своим ``work_mem``. 1000 подключений == 1000 процессов.
- › Здесь мы садим **pgbouncer**
- › Он работает как прокси - открывает указанное количество подключений к базе и передает входящие в него запросы в одно из этих подключений
- › Q: что тут может разломаться?

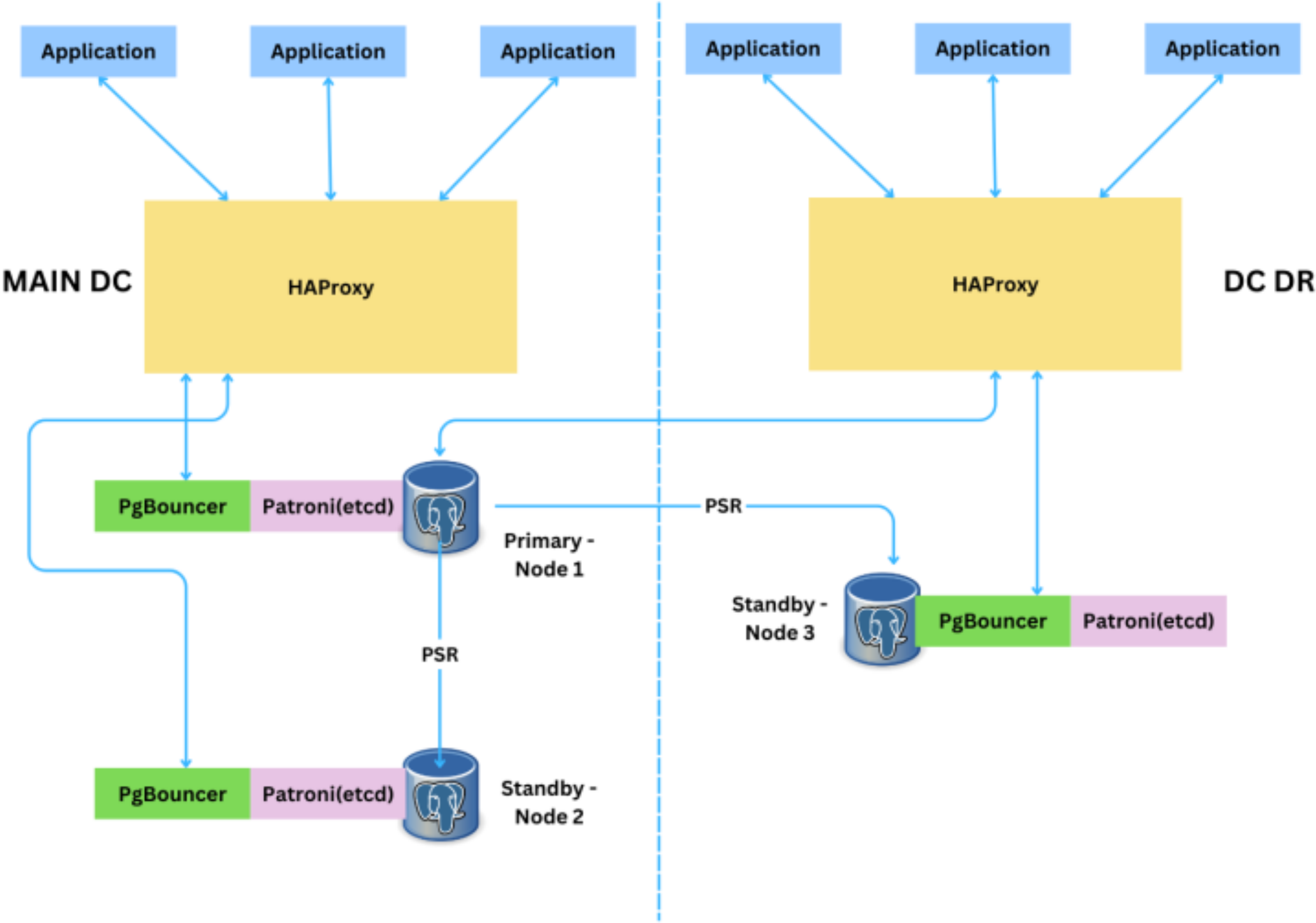
Нельзя задудосить кучей открытых подключений

- › Q: Почему `max_connections` не решает проблему?
- › A: Каждое подключение в PostgreSQL - отдельный процесс ОС со своим `work_mem`. 1000 подключений == 1000 процессов.
- › Здесь мы садим **pgbouncer**
- › Он работает как прокси - открывает указанное количество подключений к базе и передает входящие в него запросы в одно из этих подключений
- › Q: что тут может разломаться?
- › A: если отправлять стейтменты по одному, то сломаются вещи, завязанные на транзакции/сессию: временные таблицы, set-параметры, блокировки, курсоры. Для этого существуют разные режимы пула: `session`, `transaction`, `statement`

Репликация - чего еще хотим?

- › Хотим, чтобы базу нельзя было задудосить кучей открытых подключений - **pgbouncer**
- › Хотим, чтобы мастер сам переключался при смерти одного хоста - **patroni**
- › Хотим, чтобы наши запросы всегда шли в живой хост - **HAProxy**

High Availability PostgreSQL Cluster



Демо 3-4

2 - Mass Parallel Processing

Мотивация

- › Мы учимся эффективно хранить и обрабатывать большие данные
- › Наши данные перестали помещаться в один хост - дальше раздувать некуда
- › Нам не хватает вычислительных мощностей одной машины для того, чтобы обрабатывать данные с требуемой скоростью
- › А что если распараллелить вычисления, но сделать так, чтобы данные обрабатывали хосты, похожие на классические СУБД?

Мотивация

- › Мы учимся эффективно хранить и обрабатывать большие данные
- › Наши данные перестали помещаться в один хост - дальше раздувать некуда
- › Нам не хватает вычислительных мощностей одной машины для того, чтобы обрабатывать данные с требуемой скоростью
- › А что если распараллелить вычисления, но сделать так, чтобы данные обрабатывали хосты, похожие на классические СУБД?
- › Так вот оказывается, что параллельно с DFS/MR разрабатывался класс систем, которые именно это и делают - MPP

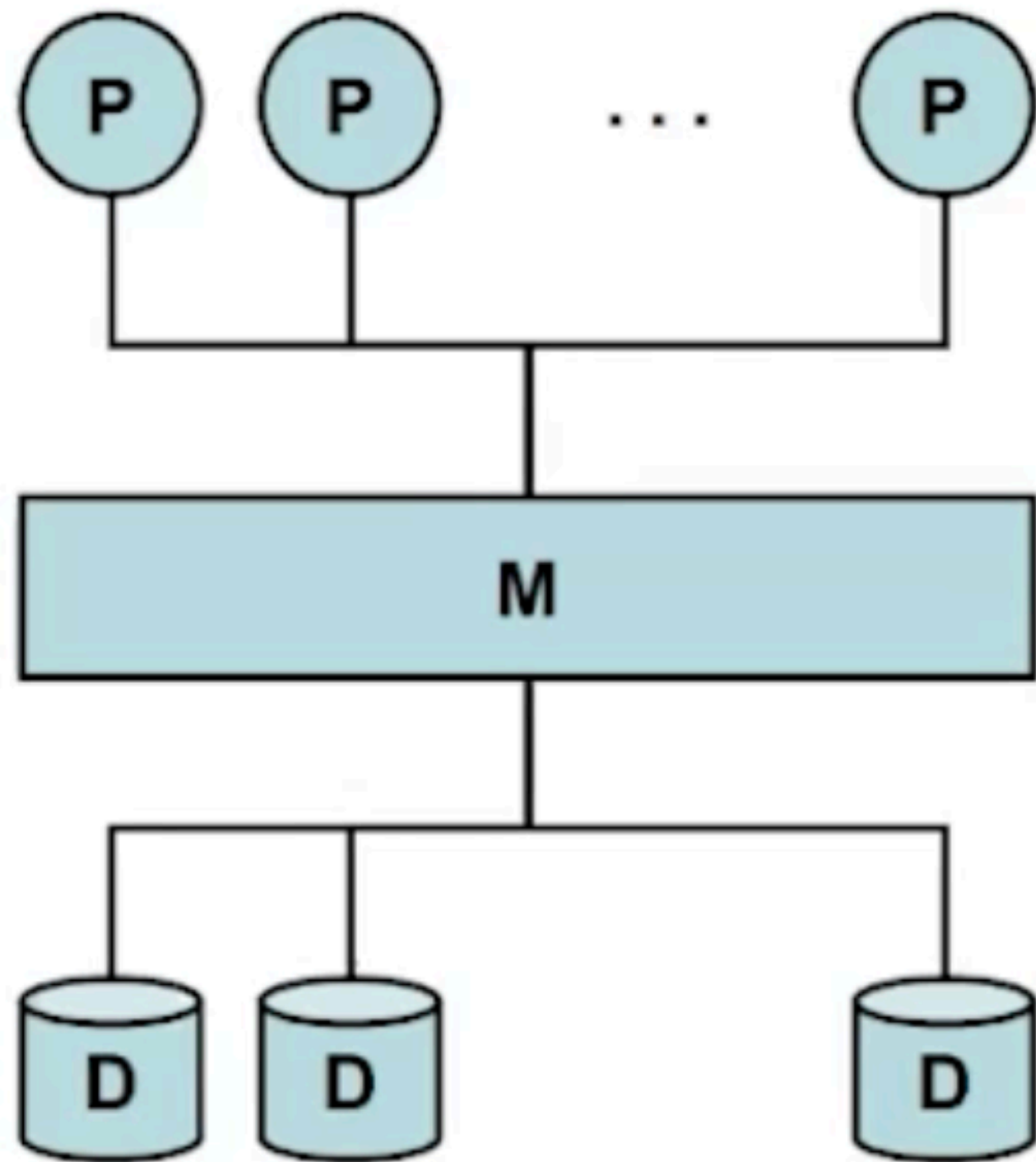
Классификация архитектур

Мы хотим создать распределенную СУБД для эффективного хранения и обработки данных.

Какие тут могут быть подходы:

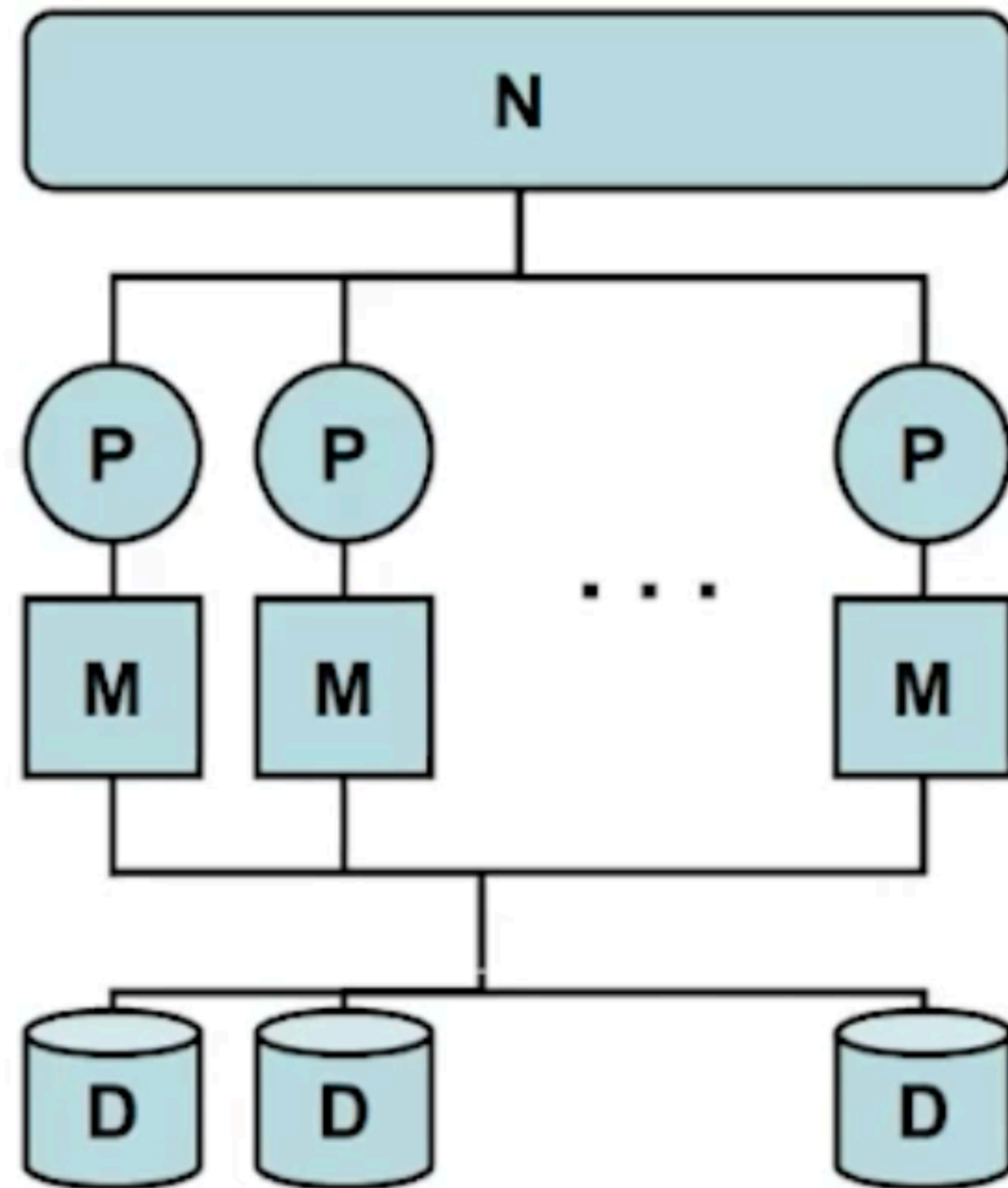
- › SE - Shared Everything - хосты делят все ресурсы (CPU, RAM, Disk)
- › SD - Shared Disks - хосты делят только диски
- › SN - Shared Nothing - нет совместного использования ресурсов
- › CD - Clustered Disk
- › CE - Clustered Everything

SE (Shared Everything)



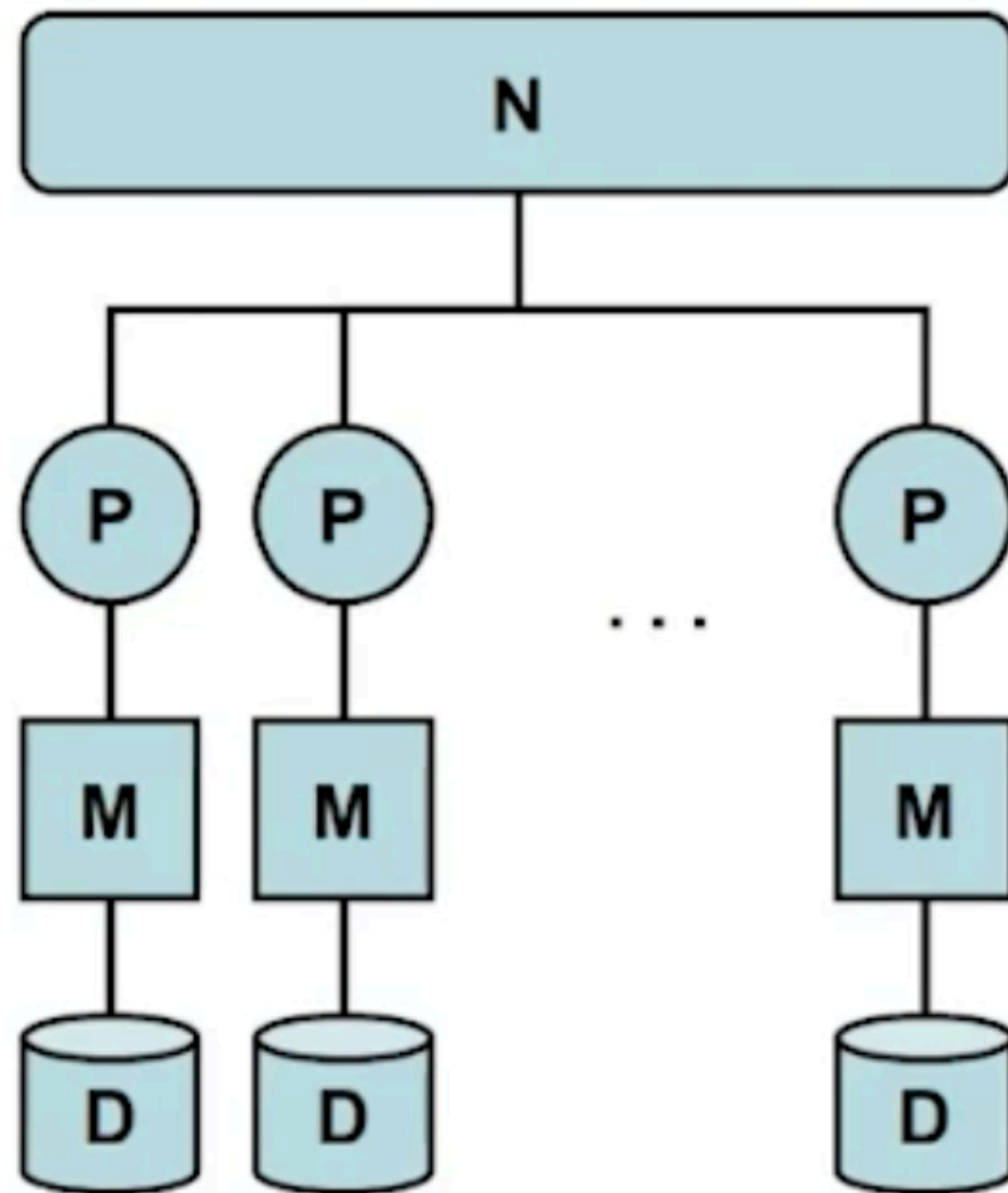
- › У всех процессоров общая RAM
- › Все диски доступны в равной степени и приоритете всем процессорам
- › Межпроцессорные коммуникации - через RAM
- › По сути - наш обычный компьютер

SD (Shared Disk)



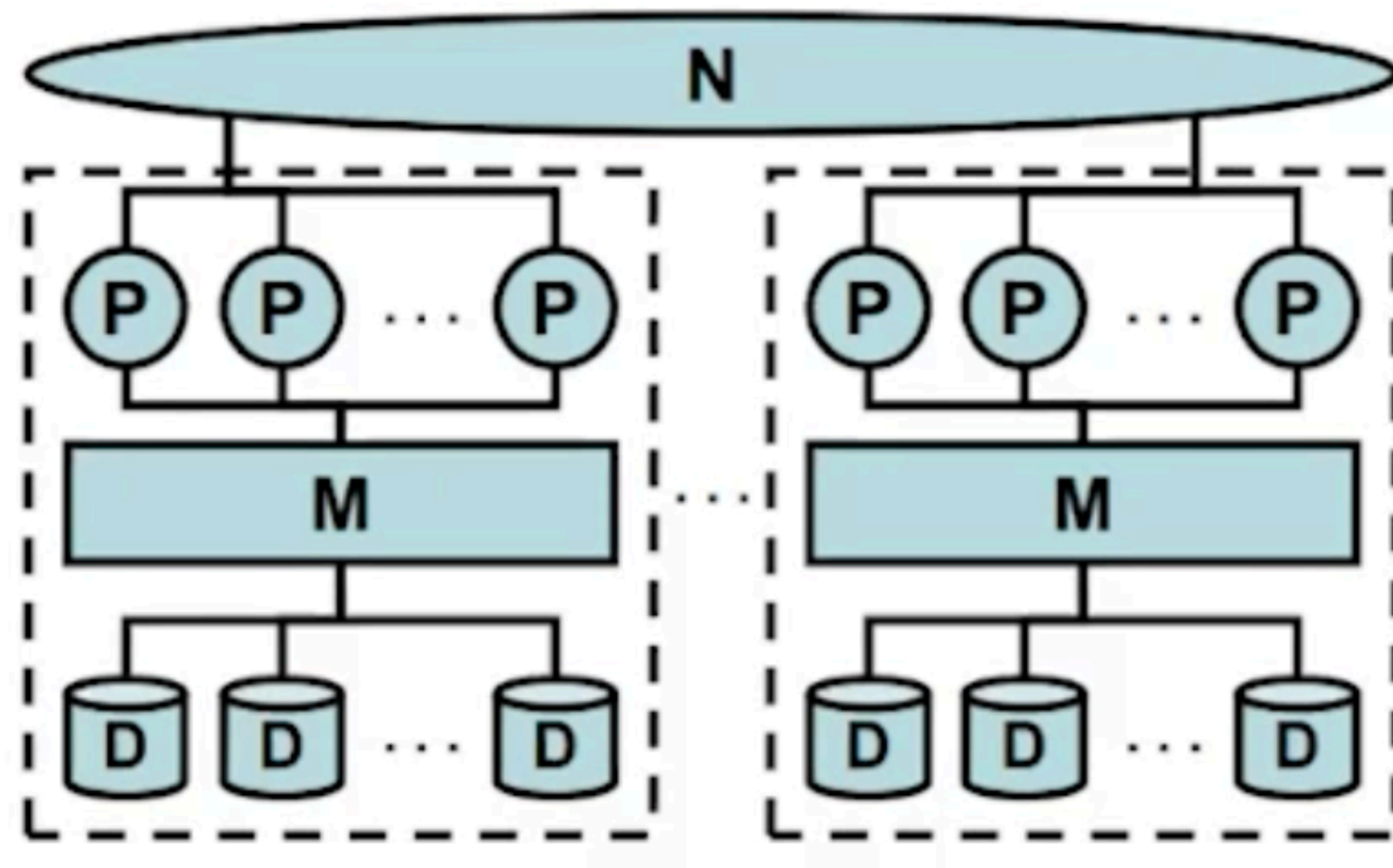
- › У каждого процессора своя RAM
- › Все диски доступны в равной степени и приоритете всем процессорам
- › Межпроцессорные коммуникации - через IO

SN (Shared Nothing)



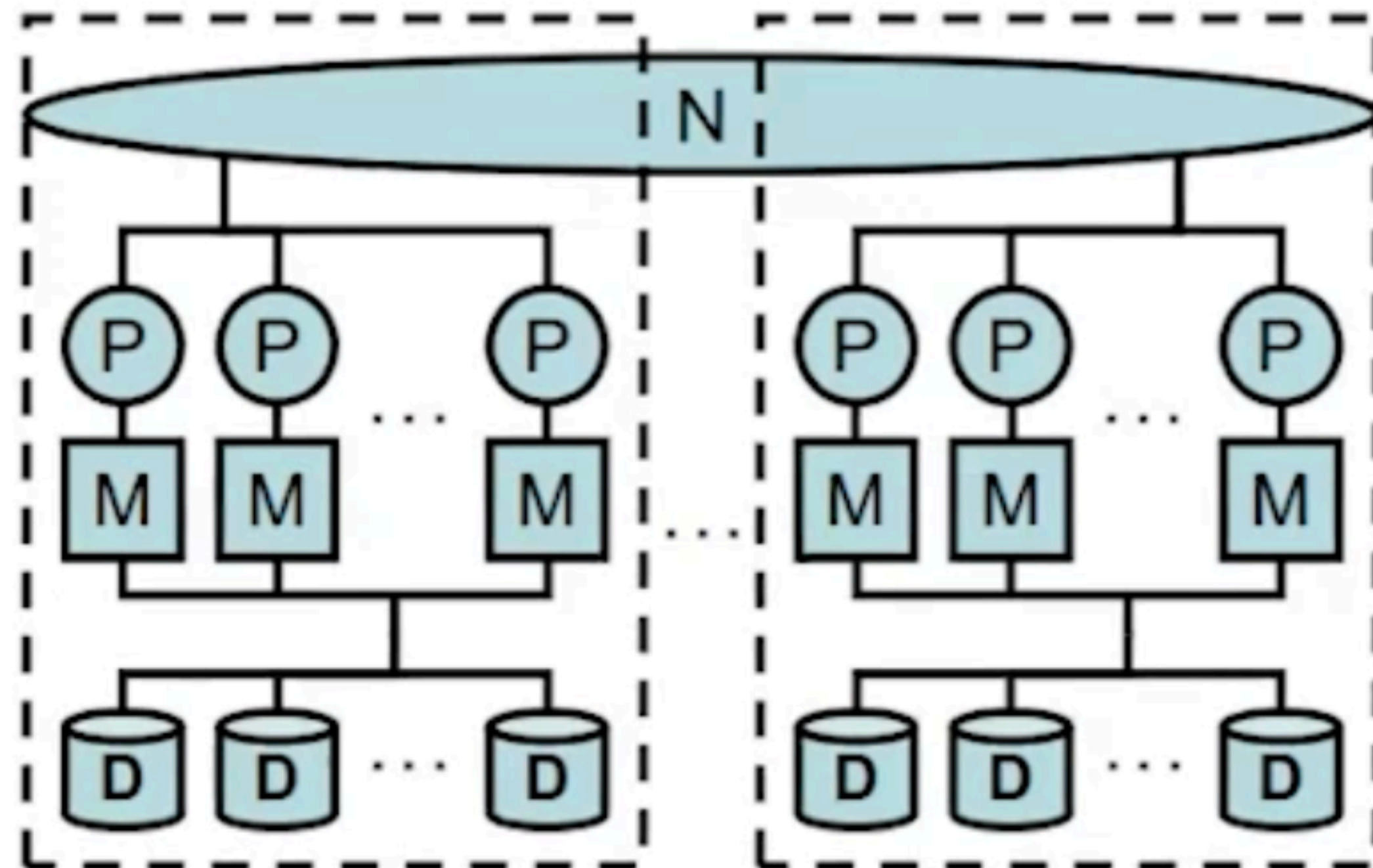
- › Каждый процессор имеет свою оперативную память и диск
- › Межпроцессорные коммуникации - через IO
- › Базовая архитектура всех MPP-расчетов: Hadoop, Vertica, GreenPlum и так далее

CE (Clustered Everything)



- › Несколько SE-кластеров, связанных сетью
- › Межпроцессорные коммуникации - через RAM
- › Межкластерные коммуникации - через IO

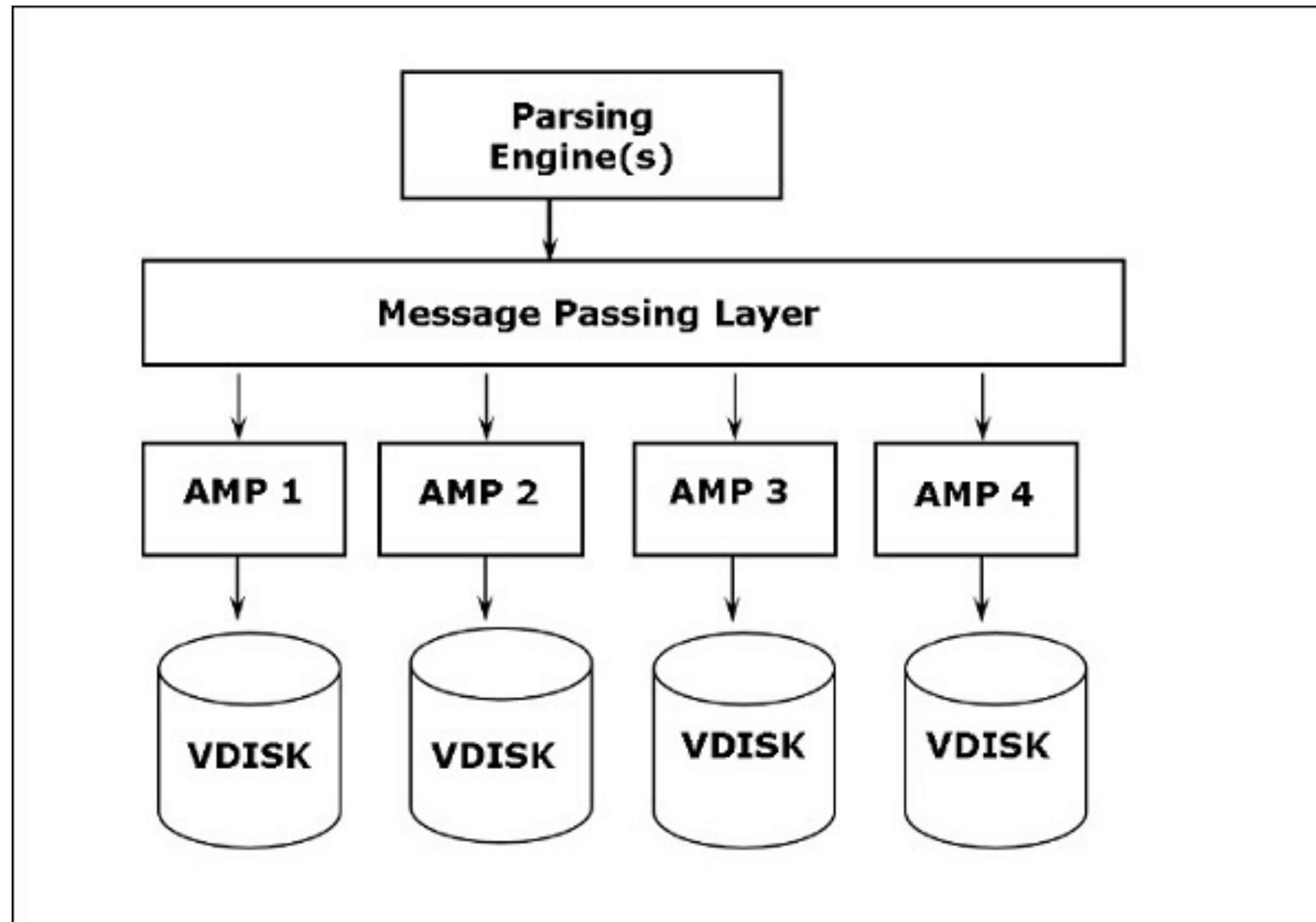
CD (Clustered Disk)



- › Несколько SD-кластеров, связанных сетью
- › Межпроцессорные коммуникации - через локальную сеть
- › Межкластерные коммуникации - через глобальную сеть

3 - Архитектура MPP

Teradata



Parsing Engines:

- › Управление сессиями пользователей
- › Чтение и оптимизация запросов
- › Выдача результатов пользователю

Message Passing Layer:

- › Посредник между PE и AMP

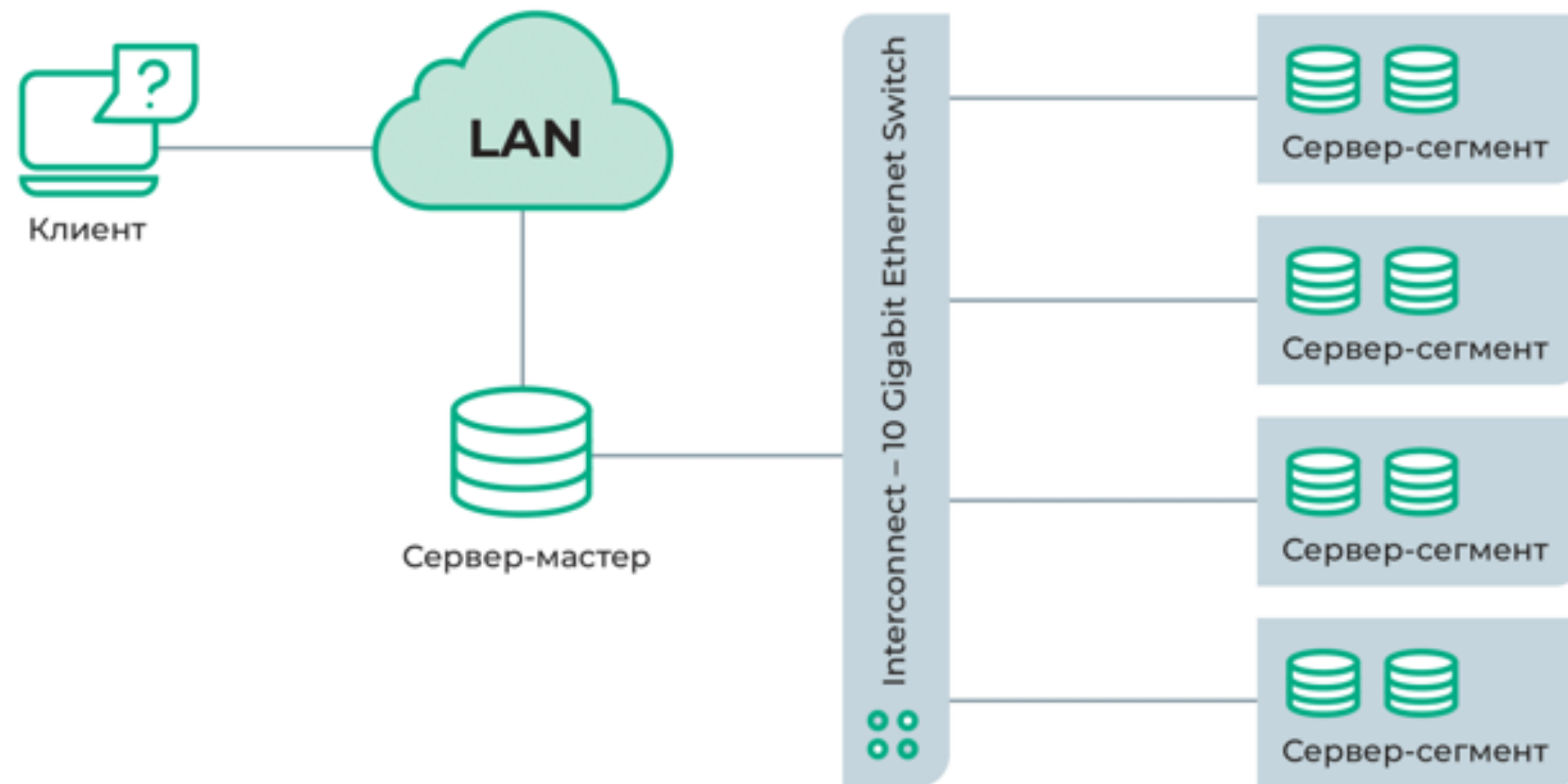
Access Module Processor:

- › Управляет выделенным пространством
- › Выполняет отправленные операции

Virtual Disks:

- › Непосредственно выделенное дисковое пространство

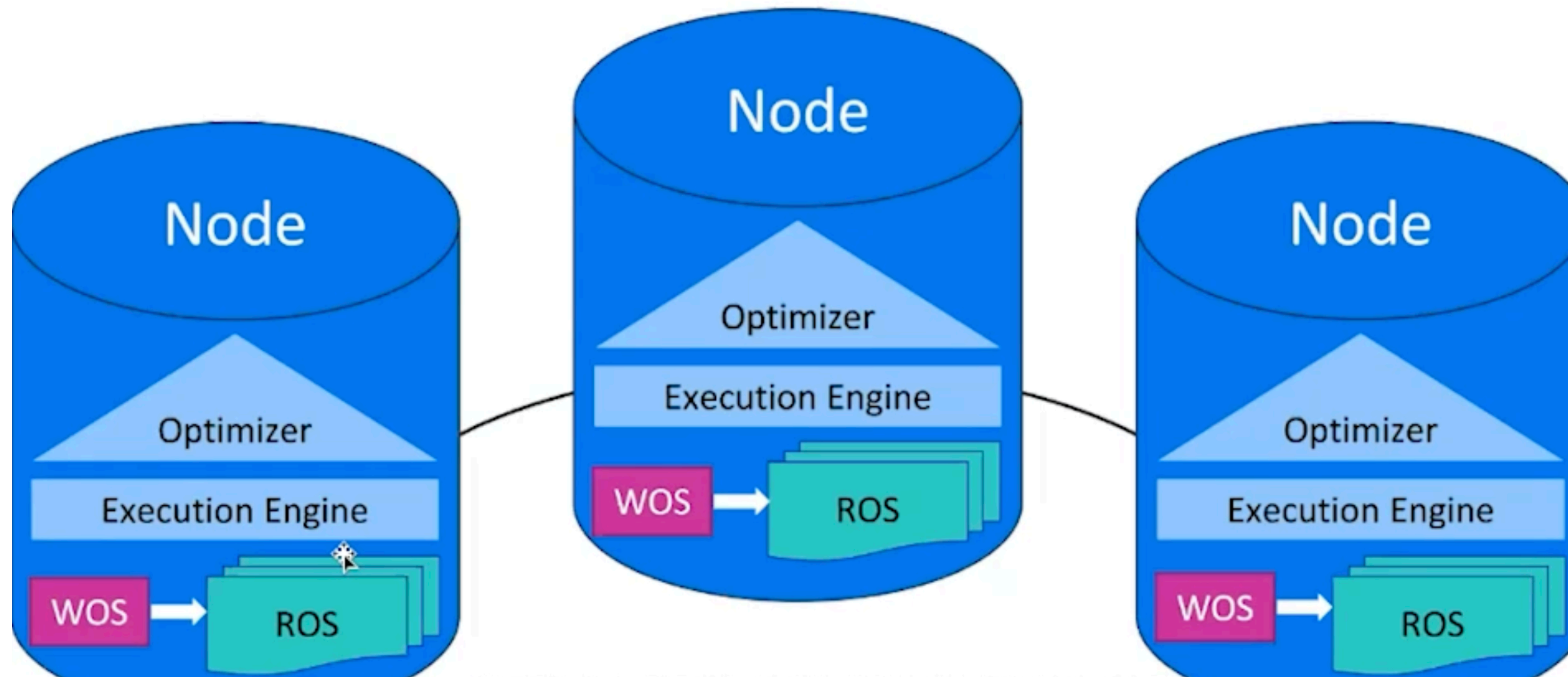
Greenplum



Pic. 3

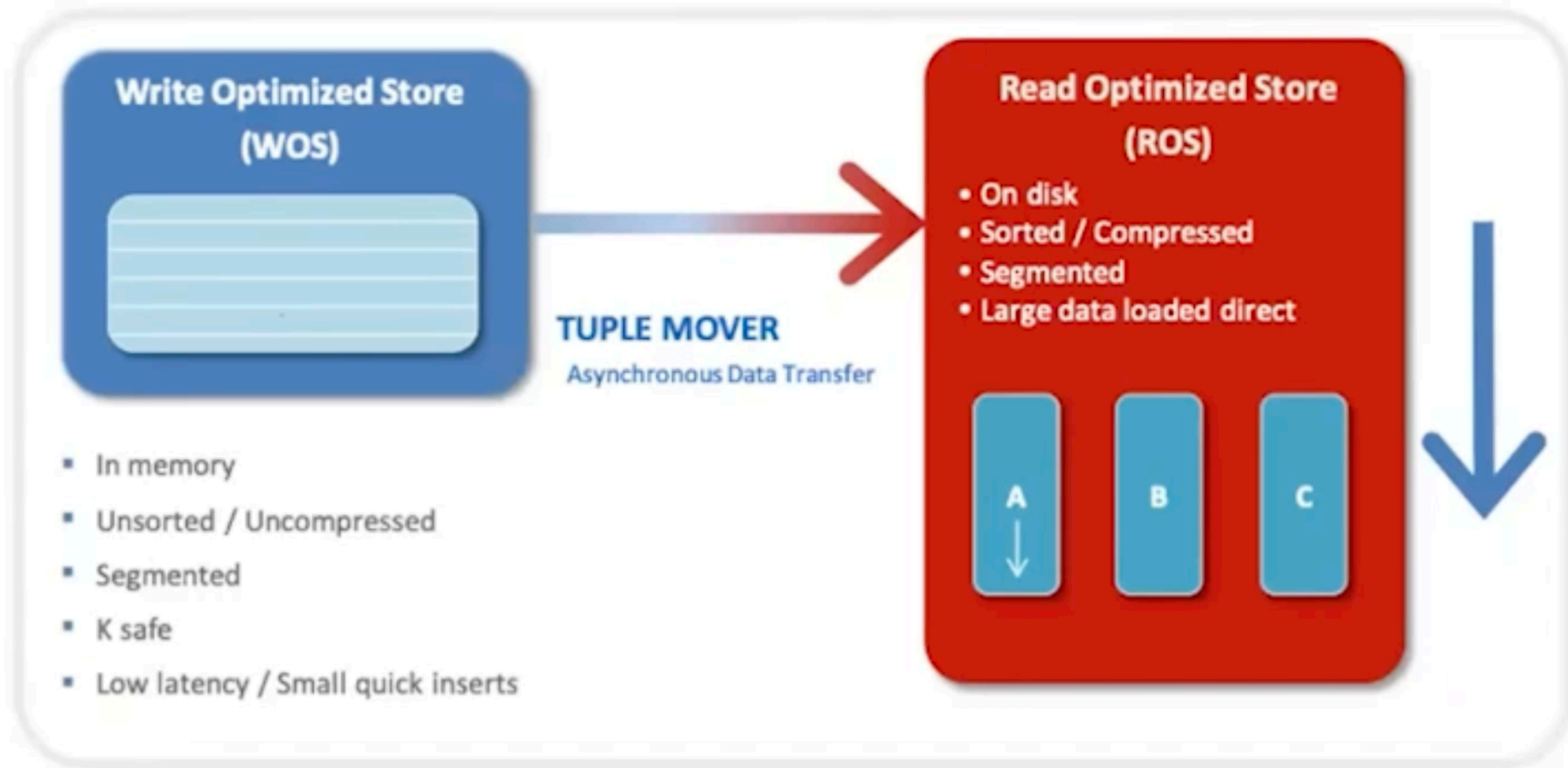
- › В GP все чуть хуже - есть одна (или две) мастер-ноды - это точка отказа и бутылочное горлышко в системе
- › Внутри сегментов по сути лежат маленькие постгресы
- › Умеет как в локальные, так и в распределенные JOIN
- › Полностью соответствует ACID
- › Нatively поддерживает постгресовые приколы (PostGIS, нативная репликация по CDC)
- › Умеет в масштабирование "на горячую"

Vertica



- › Похожа на Teradata
- › Исключительно колоночная СУБД (!)
- › Очень гибко администрируется
- › Шикарный инструмент проекций
- › WOS / ROS - память

Vertica



Фрагментарный параллелизм

Во всех перечисленных системах есть способ задать распределение по узлам:

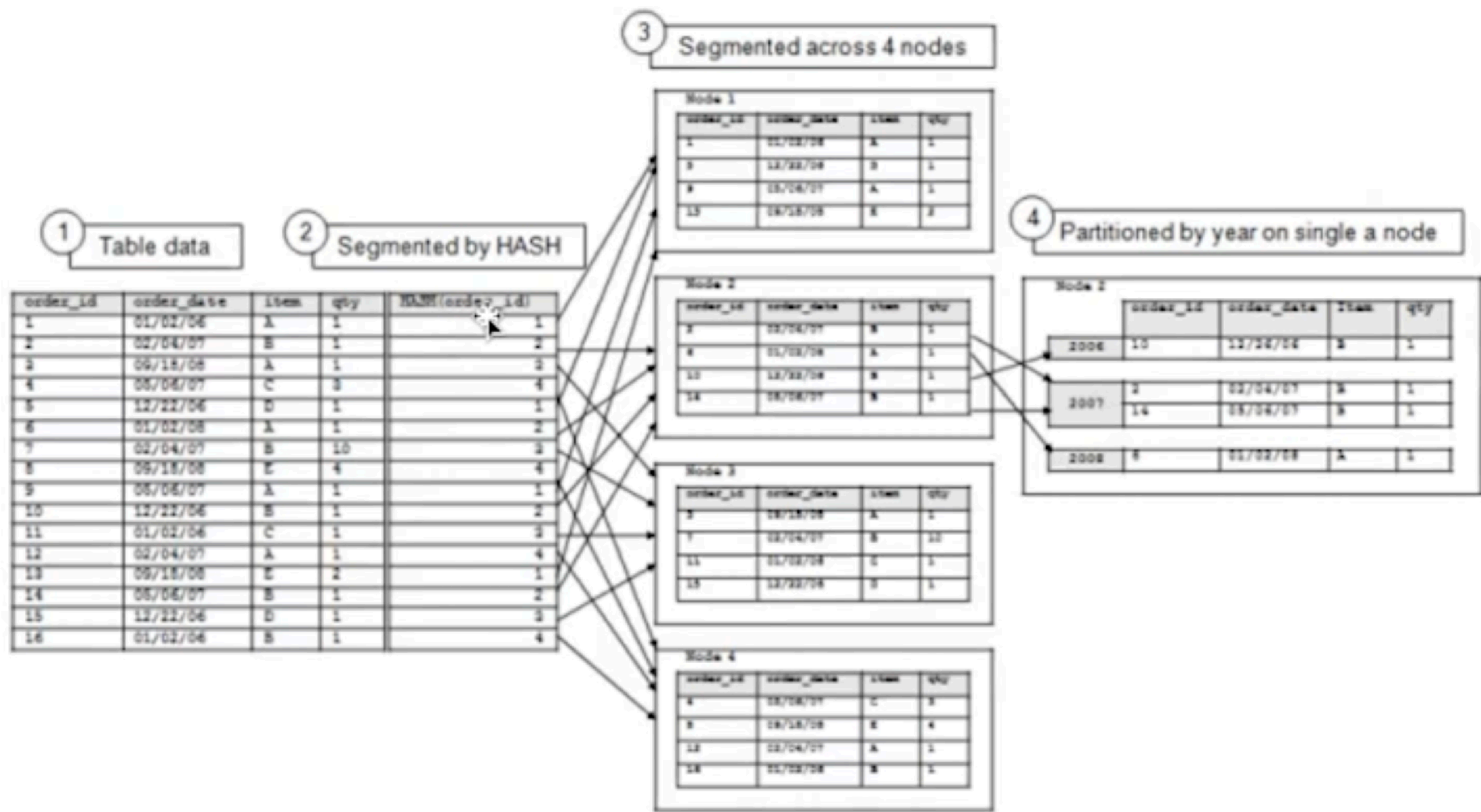
- › Teradata - primary index
- › Greenplum - distribution key
- › Vertica - segmentation key

Партицирование

Физическое разделение данных на диске по какому-то полю

- › Данные разделяются по нодам в зависимости от ключа сегментации
- › В каждой ноде данные делятся на фрагменты по ключу партицирования
- › Внутри партии строки сортируются в зависимости от выбранного ключа сортировки
- › Ключ партицирования может не совпадать с ключом сегментации

Сегментирование и партицирование вместе



Проблема выбора ключа сегментации

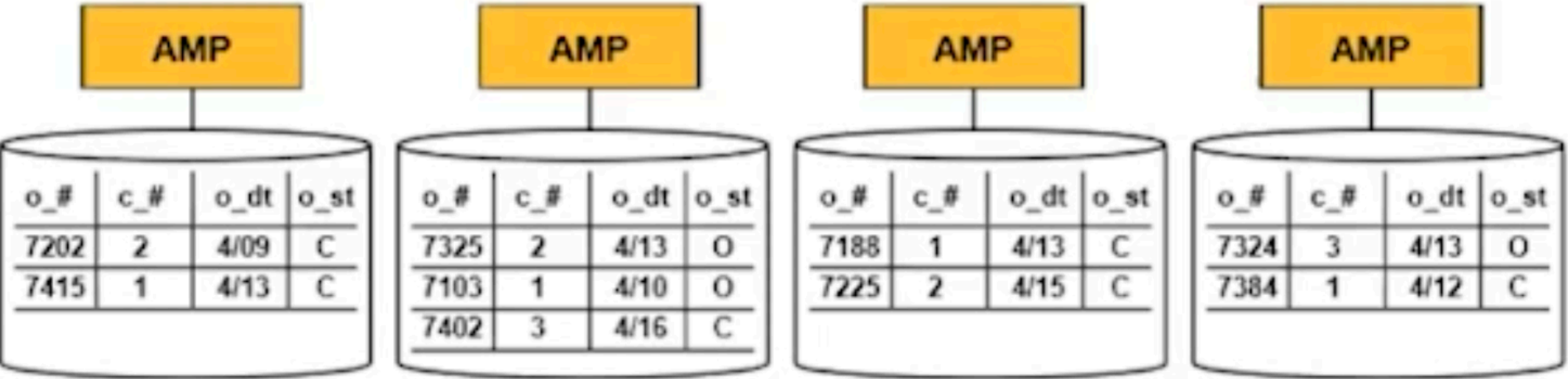
Критерии выбора ключа сегментации:

- › Доступ к данным ключа сегментации должен осуществляться чаще остальных
- › Распределение значений должно быть равномерным
- › В больших таблицах количество уникальных значений ключа должно превышать количество нод как минимум в 10 раз
- › Изменения в данных ключа сегментации не должны происходить слишком часто

Пример

Orders

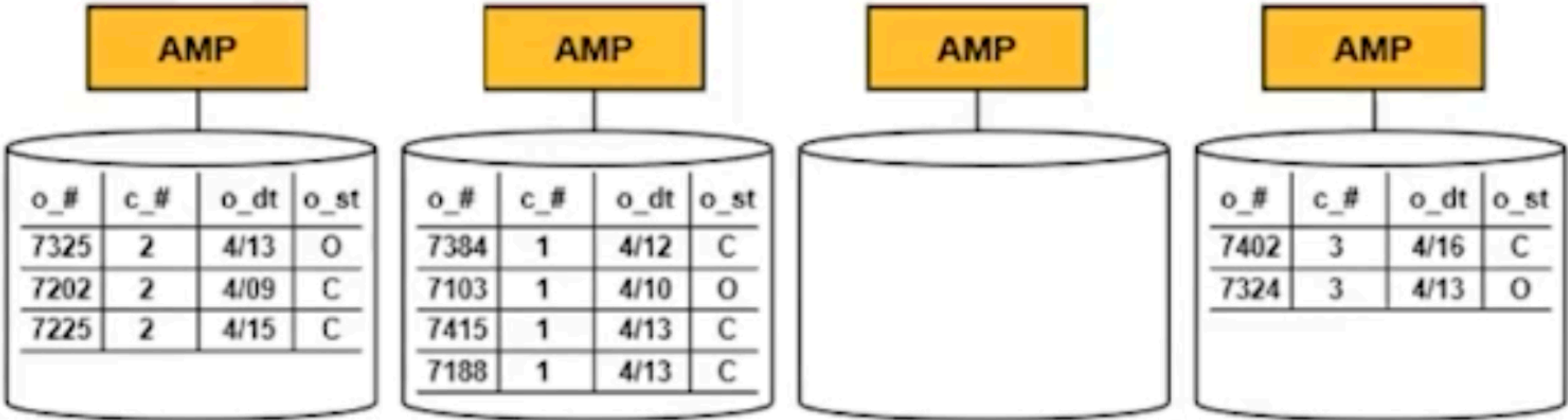
Order Number	Customer Number	Order Date	Order Status
PK			
UPI			
7325	2	4/13	O
7324	3	4/13	O
7415	1	4/13	C
7103	1	4/10	O
7225	2	4/15	C
7384	1	4/12	C
7402	3	4/16	C
7188	1	4/13	C
7202	2	4/09	C



Пример

Orders

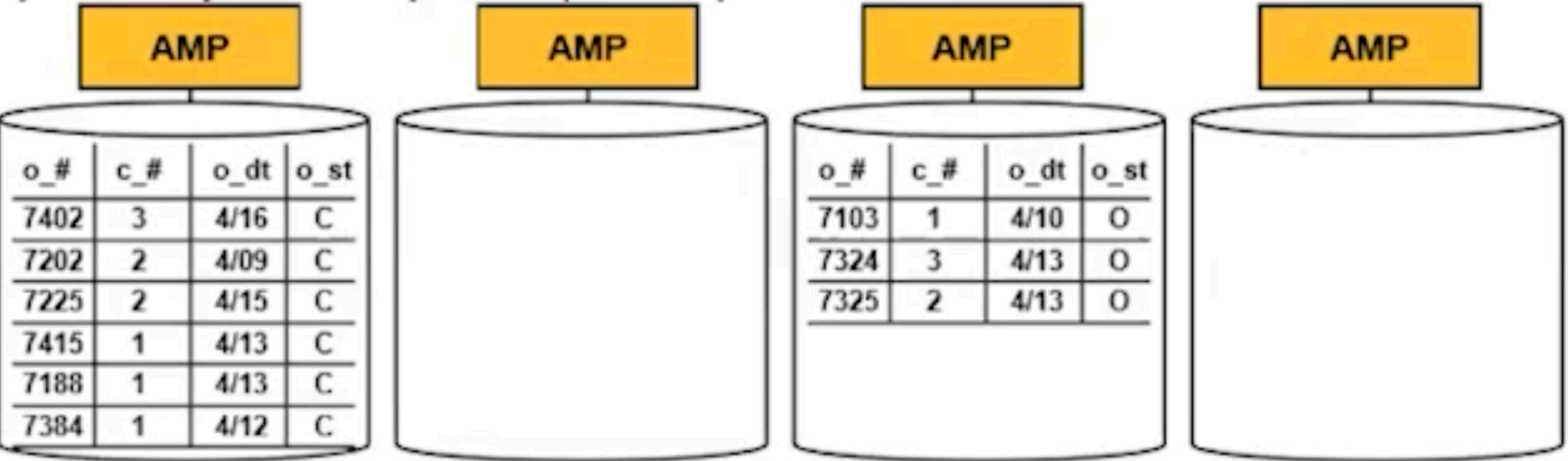
Order Number	Customer Number	Order Date	Order Status
PK			
	NUPI		
7325	2	4/13	O
7324	3	4/13	O
7415	1	4/13	C
7103	1	4/10	O
7225	2	4/15	C
7384	1	4/12	C
7402	3	4/16	C
7188	1	4/13	C
7202	2	4/09	C



Пример

Orders

Order Number	Customer Number	Order Date	Order Status
PK			
			NUPI
7325	2	4/13	O
7324	3	4/13	O
7415	1	4/13	C
7103	1	4/10	O
7225	2	4/15	C
7384	1	4/12	C
7402	3	4/16	C
7188	1	4/13	C
7202	2	4/09	C



Проблема использования партицирования

Плюсы:

- › Если на большой таблице применено партицирование, то читается только необходимый блок данных

Минусы:

- › Дополнительные накладные расходы на чтение/запись файлов
- › Запросы, которые не используют колонки партицирования, работают хуже
- › Не работает MergeJoin без использования колонок партицирования

Фрагментарный параллелизм

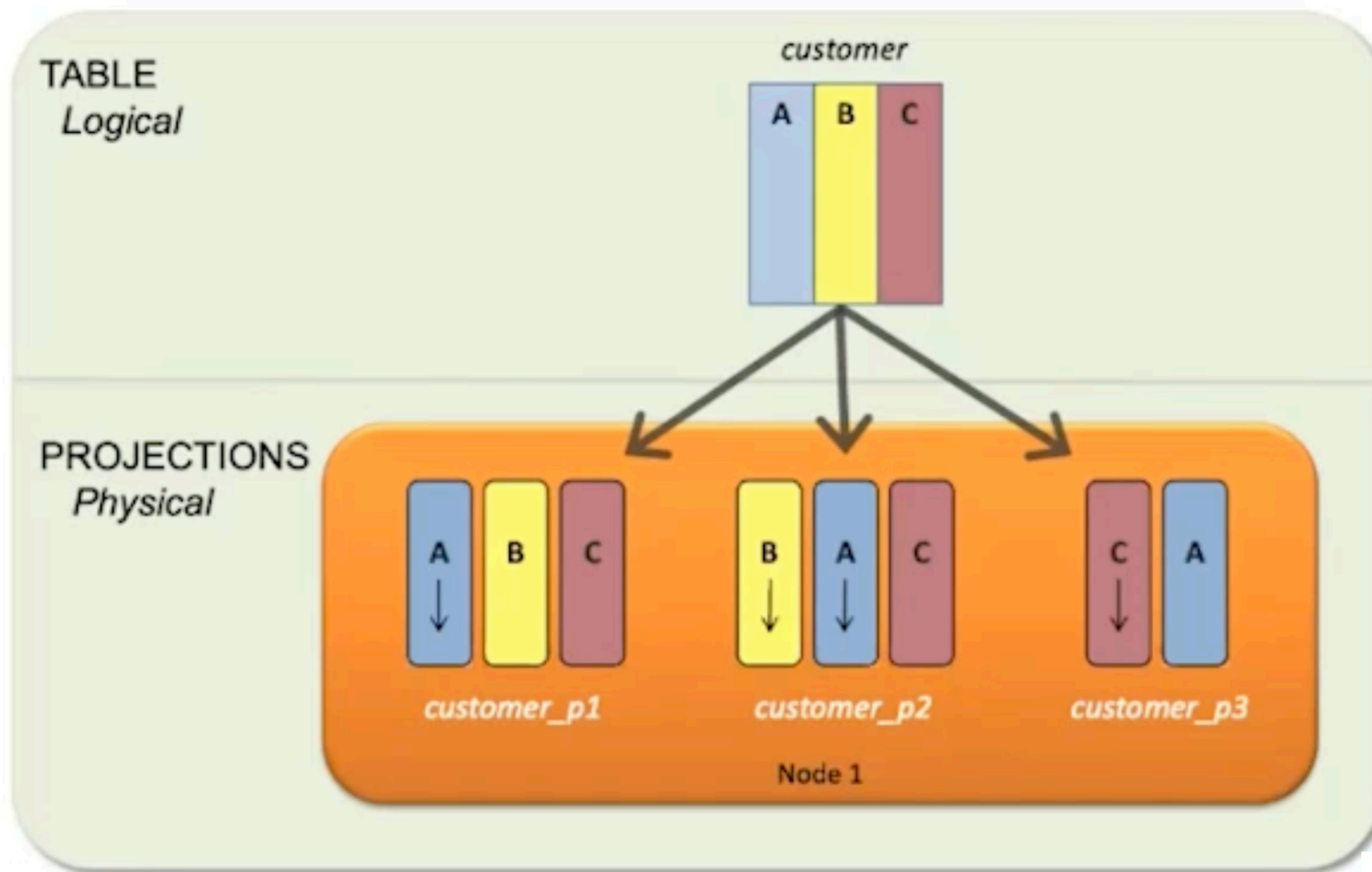
Во всех перечисленных системах есть способ задать распределение по узлам:

- › Teradata - primary index
- › Greenplum - distribution key
- › Vertica - segmentation key

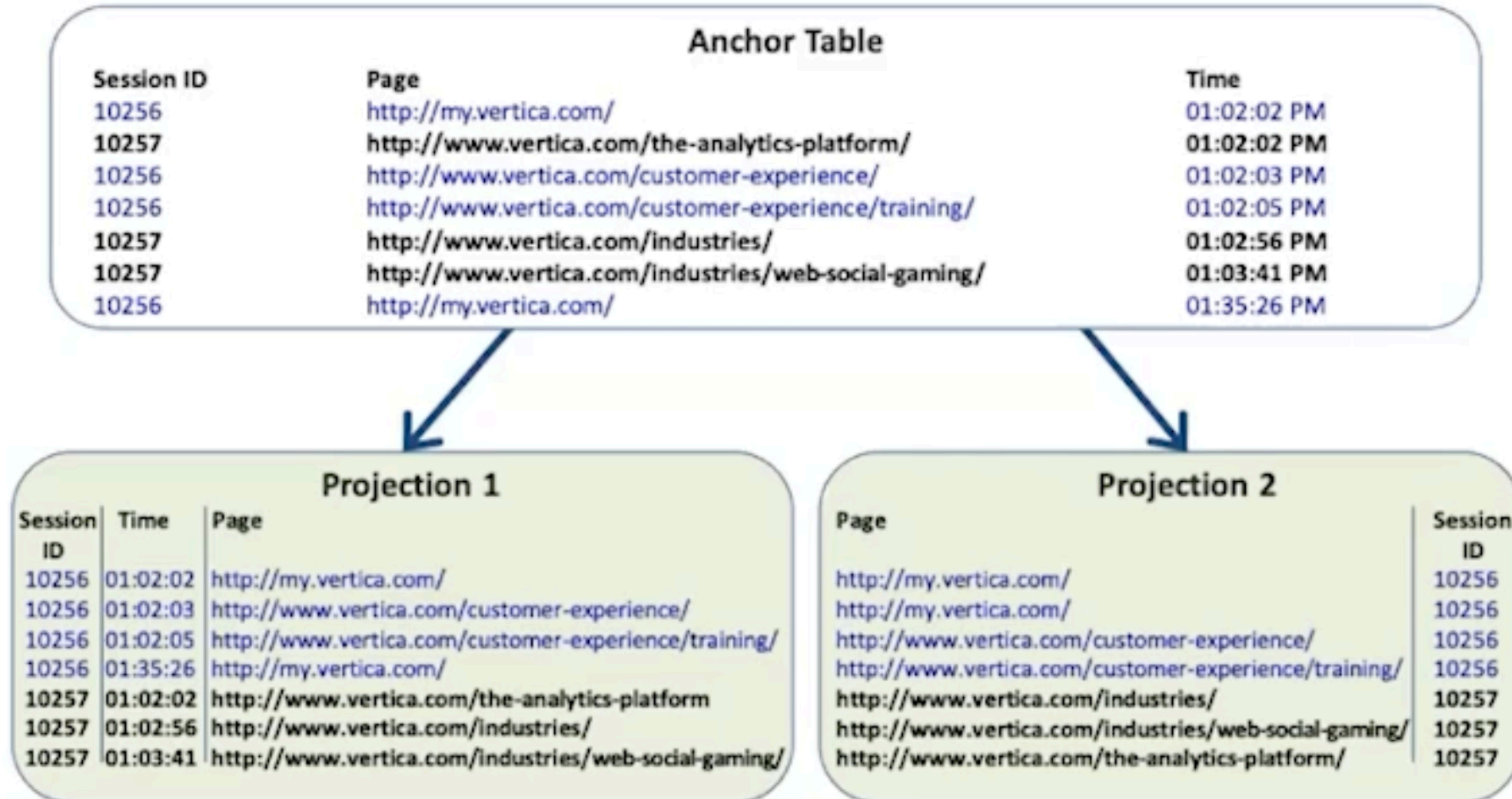
Методы доступа к данным:

- › Full scan - все MPP
- › Обращение по ключу сегментации - все MPP
- › Использование проекций - Vertica
- › Использование вторичного индекса - Greenplum

Проекции



Проекции



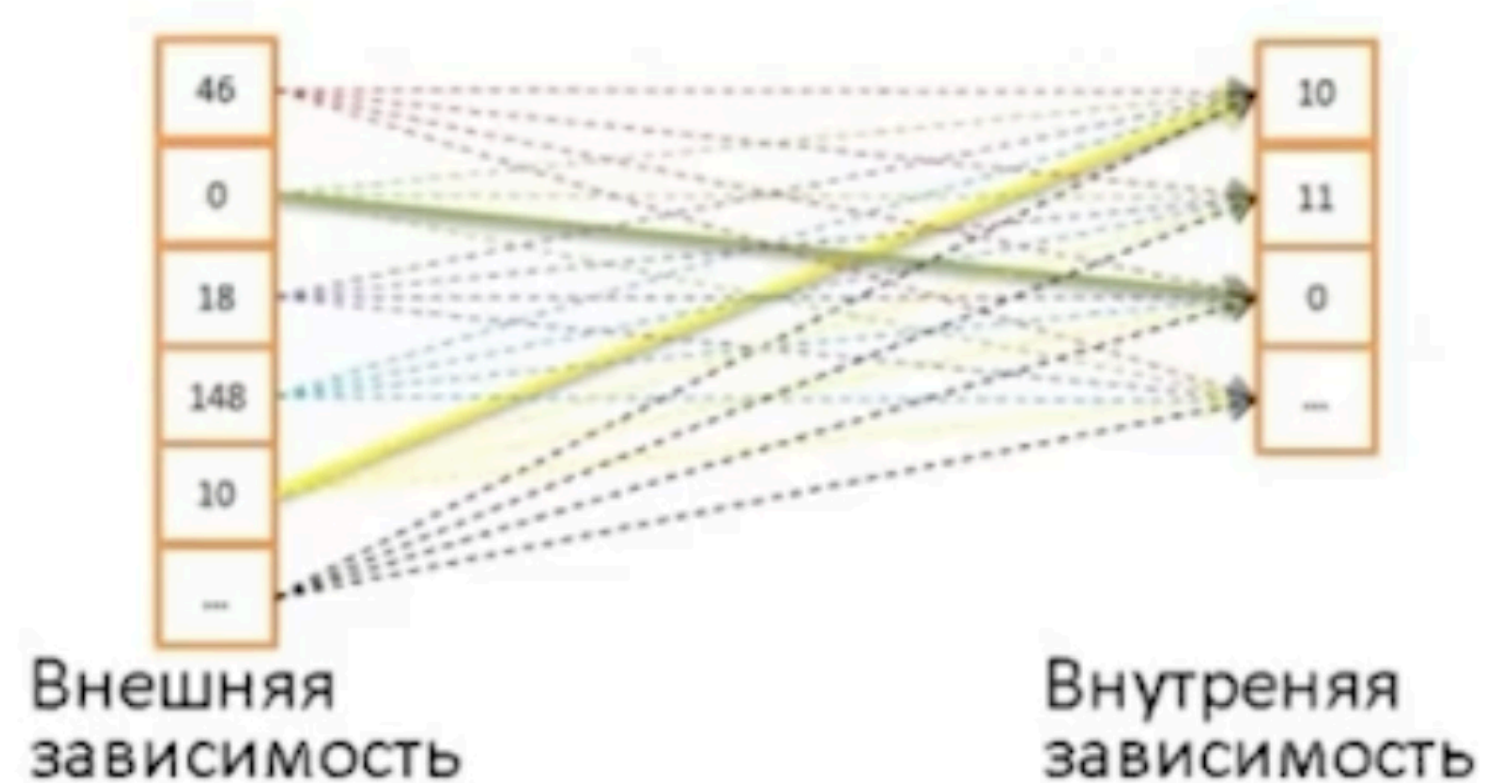
Виды JOIN

Был курс по базам?
Помним, какие виды JOIN бывают?

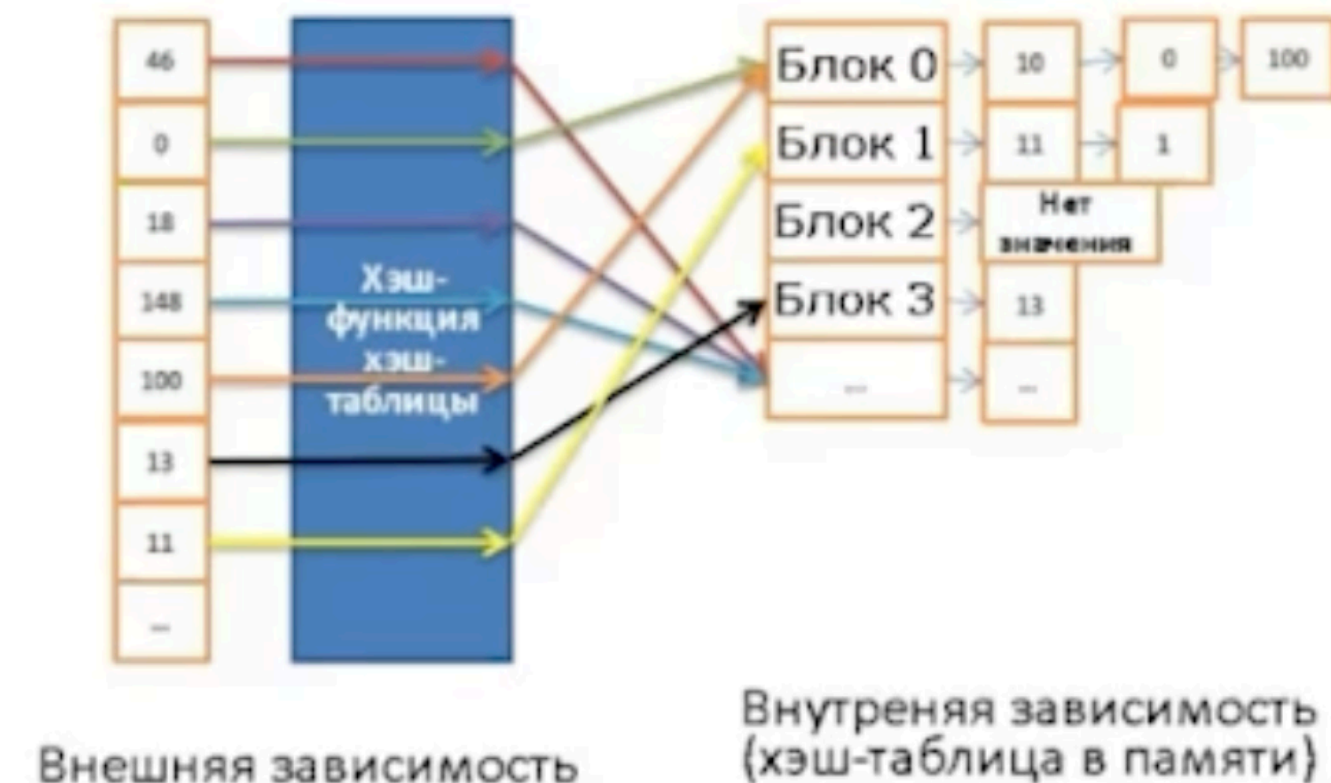


пингвин для привлечения внимания 45

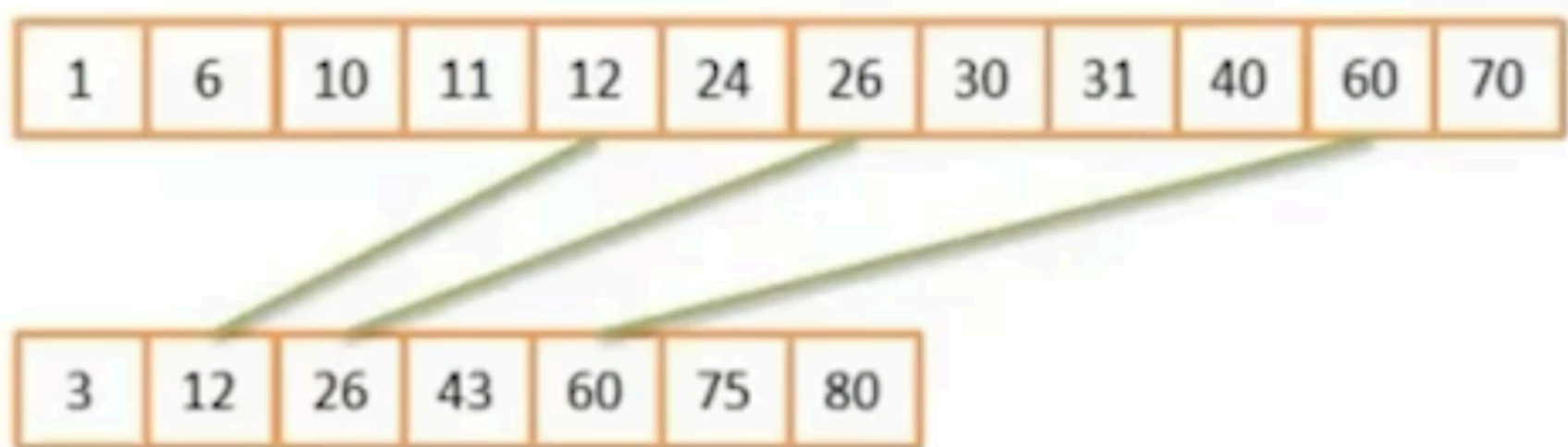
Виды JOIN



Nested loop



Hash join



Merge join

Merge join на MPP

- › Необходимо, чтобы данные были одинаково распределены
- › Блоки таблицы читаются только один раз
- › В большинстве случаев быстрее других join

Как работает на стороне БД:

- › Находим меньшее множество
- › Решаем, что с ним делать - перераспределение/дублирование/сортировка
- › Делаем join

Демо 5 - GreenPlum

That's all, Folks!

